

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

Logo
nhà trường

BÀI THỰC HÀNH 02

**Điều khiển ngữ nghĩa UR3e bằng LLM
và Skill-based Planning**

Họ và tên:	Nguyễn Văn Tổng
Mã sinh viên:	23020766
Lớp:	K68E-RE
Giảng viên hướng dẫn:	PGS. TS. Hoàng Văn Xiêm KS. Nguyễn Quốc Bảo

Hà Nội, 2026

Mục lục

1	Giới thiệu	1
2	Môi trường và kiến trúc hệ thống	1
2.1	Môi trường triển khai	1
2.2	Kiến trúc tổng thể	3
3	Thiết kế và triển khai hệ thống	4
3.1	Môi trường Gazebo và PlanningScene	4
3.2	Tích hợp Robotiq 2F-85	5
3.3	Robot skills và mô phỏng thao tác gấp	7
3.4	LLM planner, validator và executor	9
4	Cá nhân hóa nhiệm vụ và kịch bản thực nghiệm	10
5	Kết quả và đánh giá	10
5.1	Kết quả kiểm thử	10
5.2	Các vấn đề triển khai và cách xử lý	13
6	Kết luận	14
	Liên kết sản phẩm	14
A	Lệnh build, chạy và kiểm thử	15

Danh sách hình vẽ

1	Kiến trúc xử lý câu lệnh và điều khiển robot.	3
2	Các node và topic/action chính quan sát bằng rqt_graph hoặc ROS CLI.	4
3	Toàn cảnh Gazebo: UR3e gắn Robotiq 2F-85, bàn, ba cube và ba vùng A/B/C.	5
4	Mô hình UR3e + Robotiq 2F-85 và các collision object trong RViz.	5
5	Terminal hiển thị ba controller ở trạng thái active.	7
6	UR3e tiếp cận phía trên red_cube với gripper đang mở.	8
7	Robotiq 2F-85 đóng hai ngón quanh red_cube.	8
8	Cube đi theo gripper_tcp khi robot nâng hoặc vận chuyển. . .	8
9	Robot mở gripper và đặt cube vào vùng đích.	9
10	Terminal hiển thị kế hoạch JSON/skill plan sau khi LLM hoặc mock planner xử lý câu lệnh.	12
11	Terminal hiển thị các skill SUCCESS và dòng TASK SUCCESS. .	12
12	Trạng thái cuối bài nâng cao: red ở A, yellow ở B và blue ở C. .	13
13	Terminal hiển thị 20 tests, 0 errors, 0 failures, 0 skipped.	13

Danh sách bảng

1	Các thành phần phần mềm chính của hệ thống.	2
2	Pose tâm của các cube và vùng đích trong frame <code>base_link</code> . .	4
3	Các interface điều khiển chính.	6
4	Ánh xạ nhiệm vụ cho mã sinh viên 23020766.	10
5	Tổng hợp kết quả đã xác minh.	11
6	Pose cuối của kịch bản nâng cao.	12

1. Giới thiệu

Mục tiêu của bài thực hành là xây dựng một hệ thống cho phép người dùng điều khiển robot UR3e trong mô phỏng bằng câu lệnh ngôn ngữ tự nhiên tiếng Anh hoặc tiếng Việt. Thay vì yêu cầu người dùng cung cấp tọa độ hay góc khớp, hệ thống chuyển yêu cầu ở mức ngữ nghĩa thành một chuỗi thao tác đơn giản, có thể kiểm tra và tái sử dụng. Ví dụ, yêu cầu đưa khối đỏ vào vùng B được chuyển thành ba skill theo thứ tự: `pick(red_cube)`, `place(red_cube, zone_b)` và `home()`.

Nguyên tắc thiết kế quan trọng nhất là tách biệt mô hình ngôn ngữ lớn khỏi lớp điều khiển chuyển động. LLM chỉ phân tích ý định và sắp xếp các robot skill đã định nghĩa trước; LLM không được phép sinh góc khớp, pose, vận tốc hoặc trajectory. Kế hoạch do LLM trả về phải có dạng JSON và đi qua một lớp kiểm tra nghiêm ngặt trước khi bất kỳ yêu cầu chuyển động nào được gửi tới robot. Sau bước kiểm tra, MoveIt 2 chịu trách nhiệm lập kế hoạch cho cánh tay, còn bộ điều khiển gripper xử lý chuyển động đóng/mở của Robotiq 2F-85.

Cách phân tầng này tạo một ranh giới an toàn rõ ràng. Khả năng hiểu ngôn ngữ linh hoạt được đặt ở lớp trên, trong khi các hành động vật lý vẫn bị giới hạn trong một tập skill nhỏ, xác định trước và có trạng thái thực thi rõ ràng. Đây cũng là cơ sở để hệ thống có thể mở rộng thêm skill mà không cho LLM quyền điều khiển trực tiếp phần cứng.

2. Môi trường và kiến trúc hệ thống

2.1. Môi trường triển khai

Project được triển khai tại `/home/admin1/hri_ws/src/ur3_llm_control` trên Ubuntu 24.04.4, ROS 2 Jazzy, Gazebo Harmonic và MoveIt 2. Robot là UR3e gắn gripper Robotiq 2F-85. Giao tiếp LLM sử dụng API tương thích OpenAI do 9Router cung cấp. Project hỗ trợ hai chế độ planner: 9router cho LLM live và mock cho kiểm thử ngoại tuyến, trong đó cả hai chế độ đều đi qua cùng validator và cùng chuỗi thực thi robot.

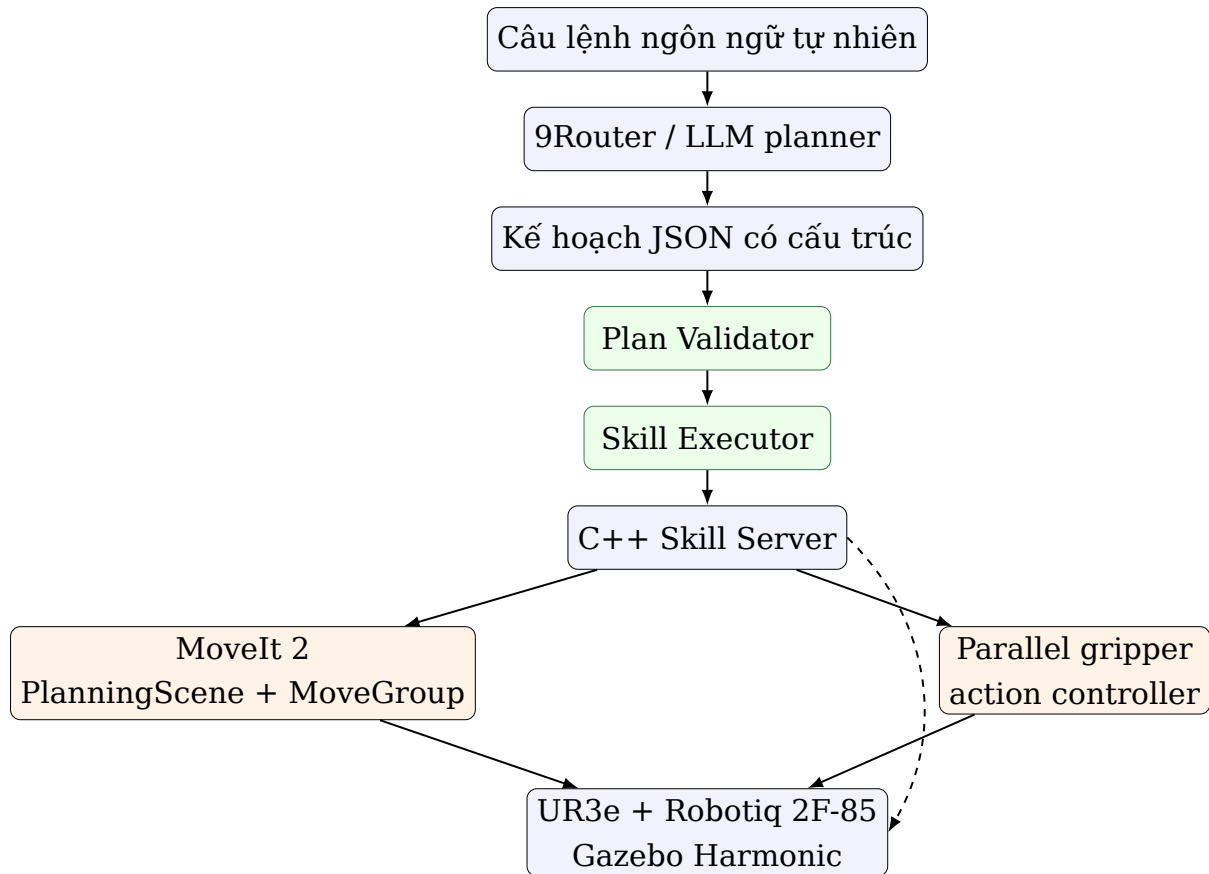
Model LLM không được hard-code trong source mà được lấy từ biến môi trường `NINEROUTER_MODEL`. File `.env.example` hiện minh họa `model gemini/gemini-3.6-flash`; model dự kiến cho buổi demo là

cx/gpt-5.6-luna. Kết quả live chỉ được công nhận sau khi ba biến NINEROUTER_BASE_URL, NINEROUTER_API_KEY và NINEROUTER_MODEL được cấu hình và có log thực nghiệm tương ứng. API key không được lưu trong repository hoặc báo cáo.

Bảng 1: Các thành phần phần mềm chính của hệ thống.

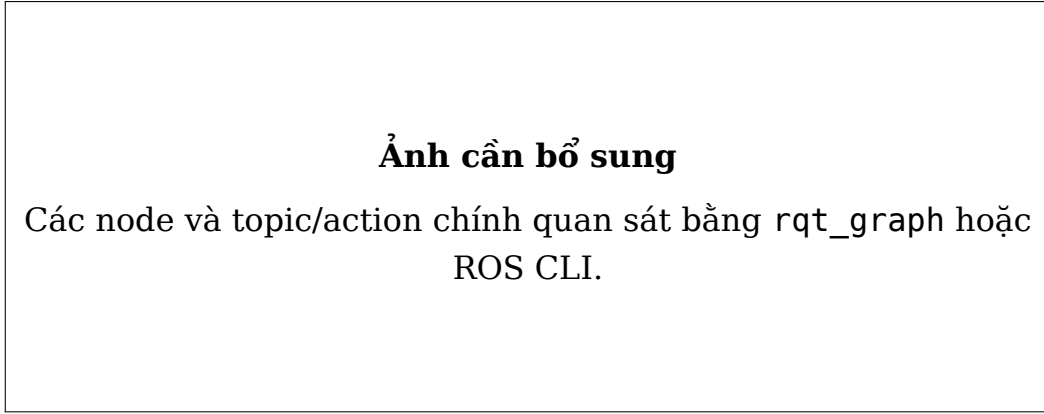
Thành phần	Vai trò
ROS 2 Jazzy	Middleware, node, service, action và topic.
Gazebo Harmonic	Mô phỏng thế giới, robot và chuyển động cơ cấu gripper.
MoveIt 2	PlanningScene, kiểm tra va chạm, IK và lập kế hoạch cánh tay.
UR3e	Cánh tay robot sáu bậc tự do.
Robotiq 2F-85	Cơ cấu kẹp hai ngón với mesh và mimic joints chính thức.
9Router/LLM	Chuyển câu lệnh tự nhiên thành kế hoạch JSON ở mức skill.

2.2. Kiến trúc tổng thể



Hình 1: Kiến trúc xử lý câu lệnh và điều khiển robot.

Node CLI nhận câu lệnh, ghép prompt hệ thống với ánh xạ nhiệm vụ cá nhân rồi gọi planner. Kết quả JSON được parse và kiểm tra schema, whitelist cũng như thứ tự trạng thái pick/place. SkillExecutor chỉ gọi skill tiếp theo khi skill trước thành công và dừng ngay khi có lỗi. Ở tầng ROS, skill_server tiếp nhận yêu cầu qua service /execute_skill, sử dụng MoveIt cho cánh tay và action /robotiq_gripper_controller/gripper_cmd cho gripper. Trong thời gian giữ vật, server đồng bộ pose của model Gazebo với frame gripper_tcp.



Hình 2: Các node và topic/action chính quan sát bằng rqt_graph hoặc ROS CLI.

3. Thiết kế và triển khai hệ thống

3.1. Môi trường Gazebo và PlanningScene

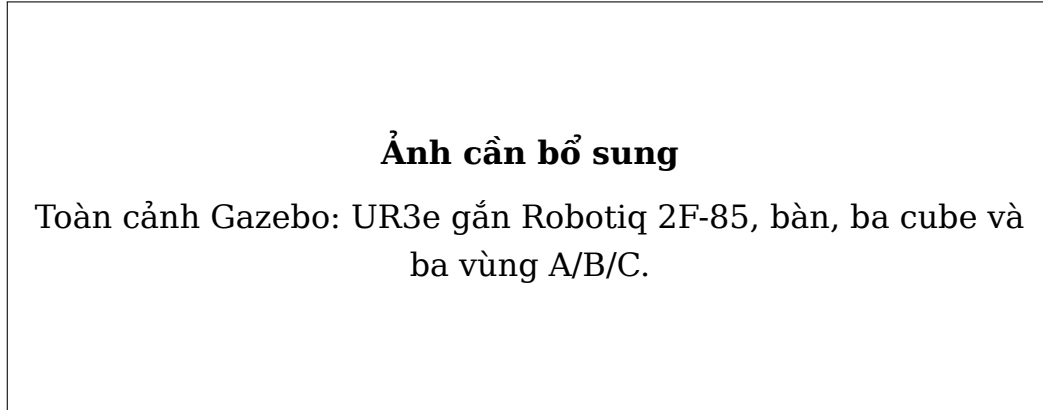
Thế giới mô phỏng gồm bàn thao tác, ba cube màu đỏ, vàng, xanh và ba vùng đích A, B, C. Bàn có tâm tại (0.47, 0, 0.03) m và kích thước $0.56 \times 0.65 \times 0.06$ m. Mỗi cube có cạnh 0.05 m. Các pose được trình bày trong Bảng 2; frame quy chiếu lập kế hoạch là `base_link`.

Bảng 2: Pose tâm của các cube và vùng đích trong frame `base_link`.

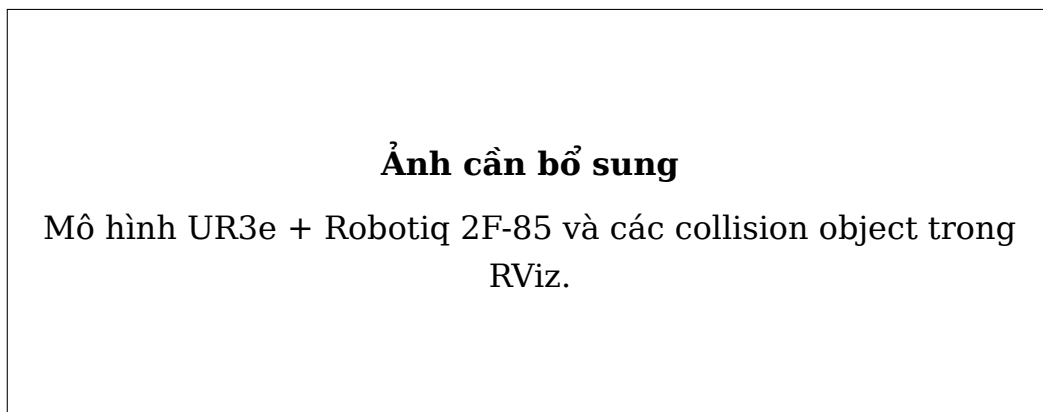
Đối tượng	x (m)	y (m)	z (m)
red_cube	0.250	-0.200	0.085
yellow_cube	0.250	0.000	0.085
blue_cube	0.250	0.200	0.085
zone_a	0.380	-0.120	0.085
zone_b	0.380	0.000	0.085
zone_c	0.380	0.120	0.085

`skill_server.cpp` đưa bàn và các cube vào MoveIt PlanningScene dưới dạng collision box. Các model cube trong Gazebo được đặt ở chế độ tĩnh và chỉ giữ phần visual. Lựa chọn này là có chủ đích: nếu cube tĩnh vẫn có collision vật lý, nó không nhượng bộ lực kẹp và làm cơ cấu mimic của 2F-85 dừng trước khi đạt góc đóng. Collision hình hộp 50 mm vẫn được MoveIt quản lý, do đó đường đi của cánh tay không bỏ qua vật cản.

File `config/scene.yaml` lưu các giá trị cảnh và thông số motion để thuận tiện theo dõi. Tuy nhiên, ở implementation hiện tại các pose tương ứng cũng được khai báo trực tiếp trong constructor của C++ skill server; vì vậy báo cáo không giả định rằng server tự động nạp toàn bộ file YAML này.



Hình 3: Toàn cảnh Gazebo: UR3e gắn Robotiq 2F-85, bàn, ba cube và ba vùng A/B/C.



Hình 4: Mô hình UR3e + Robotiq 2F-85 và các collision object trong RViz.

3.2. Tích hợp Robotiq 2F-85

Gripper sử dụng package `robotiq_description` từ repository chính thức của Robotiq [6]. Phiên bản upstream được cố định tại commit `8d7b8412ad685ffe1db5719da6e8fce6c1896e5e`. Model giữ nguyên cấu trúc link/joint và mesh hình học của nhà sản xuất. Macro adapter `ur_to_robotiq` nối `tool0` của UR3e với mount của gripper. MoveIt sử dụng semantic group `robotiq_gripper`; end-effector thao tác của cánh tay là frame `gripper_tcp`.

TCP được đặt cách base gripper 0.130 m theo trục dọc của gripper. Giá trị

này được xác định từ hình học mesh: gốc link fingertip nằm khoảng 0.098 m, trong khi pad còn kéo dài về phía trước; đặt TCP tại gốc link làm đầu ngón chạm bàn khi tâm TCP đi tới tâm cube. Vị trí 0.130 m nằm trong vùng pad hữu dụng và cho phép hai ngón bao quanh cube 50 mm.

Joint chủ động là `robotiq_85_left_knuckle_joint`; các joint còn lại chuyển động theo các quan hệ mimic trong URDF chính thức. Một hệ `gz_ros2_control` thống nhất xuất sáu joint UR và joint chủ động của gripper, tránh race giữa hai controller manager. Gazebo được khởi chạy với Bullet Featherstone vì backend này xử lý cơ cấu mimic ổn định hơn trong cấu hình hiện tại.

Bộ điều khiển gripper là `parallel_gripper_action_controller/GripperActionController` của ROS 2 Jazzy, sử dụng interface `control_msgs/action/ParallelGripperCommand`. Lệnh mở là 0.0 rad; với cube rộng 50 mm, skill server gửi góc đóng 0.35 rad. Trong kiểm thử không tải, trạng thái đo được đạt khoảng 0.330 rad khi đóng và 0.019 rad khi mở, phù hợp goal tolerance 0.02 rad.

Bảng 3: Các interface điều khiển chính.

Interface	Chức năng
<code>joint_trajectory_controller</code>	Thực thi trajectory sáu joint của UR3e.
<code>robotiq_gripper_controller</code>	Điều khiển joint knuckle qua <code>ParallelGripperCommand</code> .
<code>joint_state_broadcaster</code>	Xuất trạng thái joint cánh tay và gripper.
<code>/world/hri_assignment/set_pose</code>	Cập nhật model cube khi attach, follow và detach.

Ảnh cần bổ sung

Terminal hiển thị ba controller ở trạng thái active.

Hình 5: Terminal hiển thị ba controller ở trạng thái active.

3.3. Robot skills và mô phỏng thao tác gấp

LLM chỉ được lựa chọn ba skill `home()`, `pick(object)` và `place(object, zone)`. Skill `home()` đưa robot về named state up, là tư thế an toàn cho robot đặt cạnh mặt bàn. Các tham số vận tốc và gia tốc được giới hạn ở hệ số 0.08; OMPL với RRTConnect được sử dụng để tìm đường tránh bàn và các collision object.

Quy trình `pick` gồm: di chuyển tới pose phía trên cube; mở gripper; tạm loại collision của cube được chọn khỏi world cho đoạn hạ có chủ đích; hạ TCP; đóng ngón tới 0.35 rad; khôi phục cube và attach nó vào `gripper_tcp`; sau đó nâng robot về cấu hình phía trên. Chỉ sau khi action đóng ngón báo thành công thì object mới được attach.

Khi object đang được giữ, timer chu kỳ 50 ms đọc transform từ `base_link` tới `gripper_tcp` và gửi pose tới Gazebo. Vì vậy cube đi theo gripper liên tục trong lúc nâng và vận chuyển, thay vì chỉ nhảy vị trí tại đầu và cuối skill.

Quy trình `place` di chuyển tới phía trên zone, hạ TCP, mở gripper, dừng follow, detach object khỏi MoveIt, khôi phục collision object tại pose zone rồi rút robot lên. Đây là abstraction attach/detach dành cho simulation; hình học Robotiq, joint chủ động và chuyển động mimic của các ngón vẫn được mô phỏng thật. Giải pháp không được xem là grasp hoàn toàn dựa trên contact physics, nhưng cũng không phải thao tác teleport tức thời vì ngón kẹp đóng/mở trước attach/detach và vật đi theo TCP trong toàn bộ chuyển động.

Ảnh cần bổ sung

UR3e tiếp cận phía trên red_cube với gripper đang mở.

Hình 6: UR3e tiếp cận phía trên red_cube với gripper đang mở.

Ảnh cần bổ sung

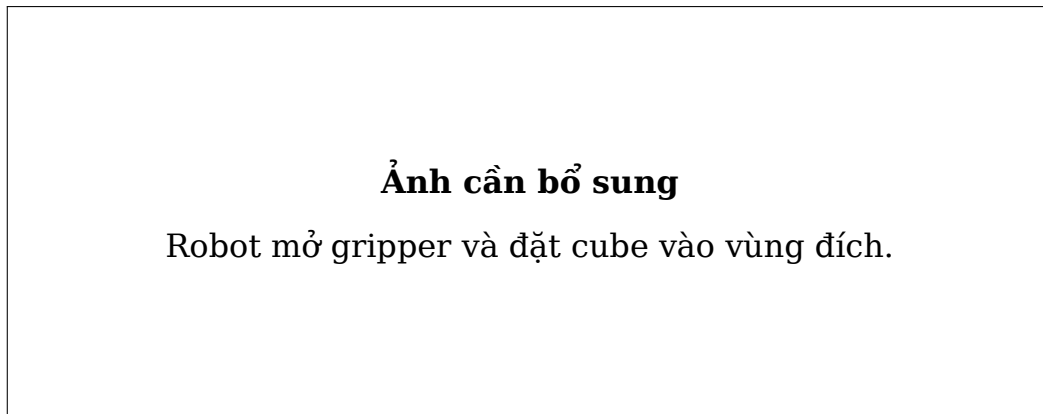
Robotiq 2F-85 đóng hai ngón quanh red_cube.

Hình 7: Robotiq 2F-85 đóng hai ngón quanh red_cube.

Ảnh cần bổ sung

Cube đi theo gripper_tcp khi robot nâng hoặc vận chuyển.

Hình 8: Cube đi theo gripper_tcp khi robot nâng hoặc vận chuyển.



Hình 9: Robot mở gripper và đặt cube vào vùng đích.

3.4. LLM planner, validator và executor

System prompt giới hạn miền hành động ở đúng ba skill, ba object và ba zone. LLM được yêu cầu trả JSON thuần, không Markdown hoặc diễn giải. Một kế hoạch hợp lệ có dạng:

```
1 {"plan": [  
2   {"skill": "pick", "object": "red_cube"},  
3   {"skill": "place", "object": "red_cube", "zone": "zone_b"},  
4   {"skill": "home"}  
5 ]}
```

Listing 1: Ví dụ kế hoạch JSON hợp lệ.

Client gửi system prompt, ảnh xạ cá nhân và câu lệnh người dùng đến endpoint `/v1/chat/completions`, với temperature bằng 0. Client chấp nhận cả JSON response thông thường và stream SSE, đồng thời retry có giới hạn đối với HTTP 429, 502, 503 và 504. Nếu thiếu cấu hình, lỗi mạng hoặc response không hợp lệ, client trả lỗi và robot không chuyển động.

Validator kiểm tra cú pháp JSON, tập field chính xác, skill/object/zone trong whitelist và trạng thái pick/place. Ví dụ, place không có pick tương ứng, green_cube, zone_d, field dư hoặc skill tùy ý đều bị từ chối trước khi gọi service robot. Sau validator, SkillExecutor chạy tuần tự và dừng tại lỗi đầu tiên. ROS service `/execute_skill` hiện dùng kiểu `rcl_interfaces/srv/SetParametersAtomically`; dữ liệu trả về có dạng `STATUS|message` để phía Python không diễn giải nội dung tùy ý.

4. Cá nhân hóa nhiệm vụ và kịch bản thực nghiệm

Hai chữ số cuối của mã sinh viên 23020766 là $XX = 66$. Chỉ số hoán vị được tính theo công thức

$$P = XX \bmod 6 = 66 \bmod 6 = 0.$$

Với $P = 0$, chương trình `student_task.py` tính được ánh xạ trong Bảng 4. Ánh xạ được sinh bởi chương trình và được đưa vào context của planner; nó không được hard-code trong prompt LLM.

Bảng 4: Ánh xạ nhiệm vụ cho mã sinh viên 23020766.

Vùng	Object	Màu
Zone A	red_cube	Đỏ
Zone B	yellow_cube	Vàng
Zone C	blue_cube	Xanh

Kịch bản cơ bản sử dụng câu lệnh “Please put the red cube in zone B.”. Kế hoạch mong đợi là `pick(red_cube)`, `place(red_cube, zone_b)` rồi `home()`. Kịch bản này kiểm tra một chu trình pick/place hoàn chỉnh, chuyển động gripper và pose cuối của cube.

Kịch bản nâng cao sử dụng câu lệnh “Arrange all objects according to my student ID.”. Với ánh xạ $P = 0$, kế hoạch gồm bảy bước: lần lượt đưa red cube vào A, yellow cube vào B, blue cube vào C, sau đó về home. `buffer_zone` có trong cấu hình và unit test trạng thái để minh họa cách phá chu trình khi các zone đã bị chiếm; tuy nhiên nó không nằm trong whitelist zone của LLM và không được dùng trong lần chạy advanced hiện tại, nơi ba cube bắt đầu từ các vị trí nguồn riêng.

5. Kết quả và đánh giá

5.1. Kết quả kiểm thử

Workspace được build thành công với năm package áp dụng cho bài thực hành, gồm Assignment 01 không thay đổi, ba package Robotiq cần cho

simulation và ur3_llm_control. Các package serial driver, hardware test và TSF camera calibration của upstream được đánh dấu COLCON_IGNORE vì không tham gia mô phỏng và serial driver yêu cầu thư viện phần cứng riêng. URDF và SRDF kết hợp được parse thành công; model chứa UR3e, adapter, toàn bộ link Robotiq, mimic joints, semantic gripper group và gripper_tcp. Lệnh test báo 20 test, 0 error, 0 failure và 0 skipped. Các test bao phủ schema validator, các plan không an toàn, cả sáu mapping MSSV, state transition, trường hợp zone bị chiếm, chu trình buffer và giải mã response 9Router.

Bảng 5: Tổng hợp kết quả đã xác minh.

Thử nghiệm	Kết quả	Ghi chú
Build workspace simulation	SUCCESS	5 package hoàn thành.
URDF/SRDF combined model	SUCCESS	Parse được model UR3e + 2F-85.
Unit/integration logic tests	SUCCESS	20/20 test đạt.
Ba controller runtime	SUCCESS	Cả ba ở trạng thái active.
Gripper close/open	SUCCESS	Đo khoảng 0.330/0.019 rad.
Basic physical simulation	SUCCESS	Thực thi bằng mock semantic planner.
Advanced physical simulation	SUCCESS	Đủ bảy skill, ba pose cuối chính xác.
Invalid plan	REJECTED	Bị chặn trước service robot.
English qua LLM live	CHƯA XÁC MINH	Cần cấu hình 9Router và lưu log demo.
Vietnamese qua LLM live	CHƯA XÁC MINH	Cần cấu hình 9Router và lưu log demo.

Kịch bản basic trong mô phỏng hoàn thành pick, place và home, đưa red cube tới (0.380, 0.000, 0.085) m. Kịch bản advanced hoàn thành cả bảy skill. Pose cuối do Gazebo báo về được liệt kê trong Bảng 6, trùng với các pose mục tiêu.

Bảng 6: Pose cuối của kịch bản nâng cao.

Object	Zone	x (m)	y (m)	z (m)
red_cube	A	0.380	-0.120	0.085
yellow_cube	B	0.380	0.000	0.085
blue_cube	C	0.380	0.120	0.085

Ảnh cần bổ sung

Terminal hiển thị kế hoạch JSON/skill plan sau khi LLM hoặc mock planner xử lý câu lệnh.

Hình 10: Terminal hiển thị kế hoạch JSON/skill plan sau khi LLM hoặc mock planner xử lý câu lệnh.

Ảnh cần bổ sung

Terminal hiển thị các skill SUCCESS và dòng TASK SUCCESS.

Hình 11: Terminal hiển thị các skill SUCCESS và dòng TASK SUCCESS.

Ảnh cần bổ sung

Trạng thái cuối bài nâng cao: red ở A, yellow ở B và blue ở C.

Hình 12: Trạng thái cuối bài nâng cao: red ở A, yellow ở B và blue ở C.

Ảnh cần bổ sung

Terminal hiển thị 20 tests, 0 errors, 0 failures, 0 skipped.

Hình 13: Terminal hiển thị 20 tests, 0 errors, 0 failures, 0 skipped.

5.2. Các vấn đề triển khai và cách xử lý

Phiên bản ban đầu chỉ có cánh tay UR3e nên thao tác gấp chưa được thể hiện. Việc tích hợp model Robotiq chính thức giải quyết phần hình học và chuyển động ngón, nhưng tạo thêm yêu cầu về controller Jazzy, mimic joints, TCP và collision semantics. Controller được chọn đúng interface ParallelGripperCommand; cánh tay và gripper được đưa vào cùng một hệ Gazebo ros2_control để tránh tình trạng đăng ký controller không ổn định.

TCP 0.098 m ban đầu trùng gốc fingertip link và làm mesh đầu ngón va chạm bàn tại pose gấp. Việc đo biên mesh cho thấy pad kéo dài xa hơn, từ đó TCP được hiệu chỉnh thành 0.130 m. Một vấn đề khác là cube tĩnh có collision vật lý làm gripper dừng khoảng 0.178 rad và action timeout. Sau khi chuyển collision cube sang PlanningScene và giữ visual-only trong Gazebo, gripper đạt mục tiêu, cube vẫn được kiểm tra va chạm khi lập kế hoạch và chỉ attach sau khi đóng ngón.

Trong lập kế hoạch cánh tay, hệ thống giữ nhánh IK gần trạng thái hiện tại cho các đoạn hạ/nâng thẳng đứng, lưu cấu hình joint phía trên cube để quay lại đúng nhánh, và unwrap wrist_3_joint quanh giá trị đo nhằm tránh sai khác tương đương 2π . Tư thế startup gập an toàn cũng ngăn model thẳng ban đầu cắt qua mặt bàn.

Ở lớp LLM, khi thiếu biến môi trường 9Router, CLI in PLAN REJECTED và không gửi skill. Đây là hành vi fail-closed mong muốn, không phải lỗi chuyển động robot. Chế độ mock cho phép kiểm tra độc lập toàn bộ validator, service và robot simulation trước khi cấu hình provider live. Các vấn đề provider, quota hoặc phiên bản Node.js chỉ nên được kết luận từ log của lần demo live cuối cùng.

6. Kết luận

Bài thực hành đã xây dựng được chuỗi xử lý hoàn chỉnh từ câu lệnh tự nhiên tới chuyển động UR3e trong Gazebo. LLM chỉ đảm nhiệm semantic planning trong miền skill giới hạn; validator kiểm tra cấu trúc và trạng thái; MoveIt 2 lập kế hoạch cánh tay; còn action controller điều khiển Robotiq 2F-85. Hai nhiệm vụ basic và advanced đã được thực thi thành công trong mô phỏng với mock semantic planner, đồng thời pose cuối của ba cube khớp các vùng cá nhân hóa theo mã sinh viên.

Kiến trúc skill-based giúp hành vi robot dễ kiểm tra, tái sử dụng và mở rộng hơn so với việc để LLM sinh lệnh joint trực tiếp. Hạn chế chính là thao tác giữ vật dùng abstraction attach/detach thay vì contact grasp hoàn toàn vật lý; ngoài ra kết quả LLM live phụ thuộc cấu hình và trạng thái dịch vụ 9Router. Hướng phát triển tiếp theo là bổ sung perception, phản hồi grasp, quản lý buffer ở runtime và kiểm thử live có lưu log tái lập được.

Liên kết sản phẩm

- GitHub repository: **[ĐIỀN LINK GITHUB]**
- Video demo: **[ĐIỀN LINK VIDEO]**
- Model LLM của demo live: **[XÁC NHẬN MODEL SAU KHI CHẠY]**

- Log LLM live tiếng Anh/tiếng Việt: **[BỔ SUNG SAU KHI CẤU HÌNH 9ROUTER]**

Tài liệu

- [1] Open Robotics, *ROS 2 Jazzy documentation*, <https://docs.ros.org/en/jazzy/>.
- [2] MoveIt, *MoveIt 2 documentation*, <https://moveit.picknik.ai/main/index.html>.
- [3] Open Robotics, *Gazebo Harmonic documentation*, <https://gazebo.sim.org/docs/harmonic/>.
- [4] ros-controls, *ros2_control documentation for Jazzy*, <https://control.ros.org/jazzy/index.html>.
- [5] Universal Robots, *Universal Robots ROS 2 Description*, https://github.com/UniversalRobots/Universal_Robots_ROS2_Description.
- [6] Robotiq, *Official ROS repository for Robotiq grippers*, <https://github.com/robotiq/ros>.

A. Lệnh build, chạy và kiểm thử

```
1 cd /home/admin1/hri_ws
2 source /opt/ros/jazzy/setup.bash
3 colcon build --symlink-install
4 source install/setup.bash
5 colcon test --packages-select ur3_llm_control
6 colcon test-result --verbose
```

Listing 2: Build và kiểm thử workspace.

```
1 # Terminal 1
2 ros2 launch ur3_llm_control llm_robot.launch.py
3
4 # Terminal 2
5 ros2 run ur3_llm_control task_cli --mode mock \
```

```
6 "Please put the red cube in zone B."
```

Listing 3: Khởi chạy mô phỏng và kiểm thử không cần API key.

```
1 export NINEROUTER_BASE_URL="http://localhost:20128/v1"  
2 export NINEROUTER_API_KEY="<YOUR_KEY>"  
3 export NINEROUTER_MODEL="cx/gpt-5.6-luna"  
4 ros2 run ur3_llm_control task_cli
```

Listing 4: Cấu hình LLM live qua 9Router; không ghi key thật vào báo cáo.